

User-Triggered UAV Dispatching System Technical Guide

This guide explains how to set up and operate the Android mobile application, Windows ground station and onboard UAV software. It also describes the three-mode MATLAB simulation, saved results and original system imagery.

1 Project Contents

User-Triggered UAV Dispatching System | Technical Guide | 1 October 2026
Implementation baseline: UAV-SEARCH-20260928

This archive contains the mobile application, ground station and UAV source code, together with MATLAB simulations of the three search modes and outdoor footage. All paths in this guide are relative to the archive root.

File locations

Directory	Contents
code/	Mobile application, ground station and onboard UAV software
simulation/	MATLAB scripts, elevation data, trajectories, figures and animation
docs/assets/outdoor/	Original outdoor footage, ground-station footage and images
docs/	This guide in Word, PDF and Markdown formats
LICENSES/	Source licenses and third-party copyright notices

Component directory abbreviations

M, G and U refer to the mobile, ground-station and UAV directories, respectively:

code/mobile_application
code/ground_station
code/search_uav

Current behavior

Mode 1 estimates the trip end time from the planned route and a fixed walking speed. Mode 2 checks whether the latest valid GPS sample exceeds the configured update timeout. Mode 3 places an SOS request in the verification queue. In all three modes, the operator verifies the situation and confirms a search before creating an event, reviewing its route and dispatching the mission.

The software retains its English and Chinese interface options. Section 8 maps the system functions to their implementation files.

2 Component Setup

Use Node.js 24 and the Android SDK/JDK for the mobile application, Python 3.12 with Tk support on Windows for the ground station, and Ubuntu 20.04 with ROS Noetic for the UAV. MATLAB R2025b was used for the saved simulation.

Android application

From code/mobile_application, install the locked dependencies with npm ci. Copy src/services/db/firebaseConfig.example.ts to src/services/db/firebaseConfig.ts and fill the deployment values.

Supply the native Android service configuration and map credentials. Start Metro with `npm start` and run `npm run android` in another terminal. The mobile guide describes the project structure and setup.

Windows ground station

From the repository root in PowerShell, create and activate a Python environment:

```
py -3.12 -m venv .venv
.\venv\Scripts\Activate.ps1
python -m pip install -r code\ground_station\requirements.txt
```

Fill `code/ground_station/config/runtime.env` using the adjacent example, then export its settings before launching the application. Section 4 gives the runtime settings and launch commands. To produce the executable, install `code/ground_station/packaging/requirements-build.txt` and run `code/ground_station/packaging/build_windows_release.ps1`.

Onboard software and simulation

Follow Flight environment setup for the external flight workspace and ROS dependencies. Build the flight workspace before `rescue_bridge`. Section 5 describes the installed components; Section 6 gives the MATLAB entry points and scenario parameters.

3 Mobile Records and Trip-Time Estimation

Mode 1 Event Booking

The user enters a trip name, departure date and departure time, then selects successive locations along the planned route on the map. The interface displays the estimated end date and time automatically. Editing the route, departure date or departure time recalculates the estimate.

The application uses the Haversine formula between consecutive coordinates and sums these distances. It uses a fixed walking speed of 4 km/h and rounds the estimated duration up to a whole minute. The end timestamp is the departure timestamp plus that duration. Dates spanning midnight, a year boundary or a leap day retain the complete end timestamp.

```
duration_minutes = ceil(route_distance_m / 4000 * 60)
end_time = departure_time + duration_minutes * 60 seconds
```

The 4 km/h value follows the general walking-time reference in Ordnance Survey's Map Reading guide. This estimate uses horizontal route distance; selected points should represent the planned route turns. Gradient, ground conditions and stops affect actual duration, so an overdue record still requires operator contact and verification.

Stored field	Meaning
waypoints	Planned route positions in the order selected by the user
expectedEndAtMs	Estimated end timestamp in Unix milliseconds
endDate / endTime	End date and time displayed by the interface
routeDistanceM	Total route distance in metres
estimatedDurationMinutes	Estimated duration, rounded up to whole minutes
walkingSpeedKmh	Fixed value: 4
estimationMethod	route_distance_fixed_speed_v1

Mode 2 Quick Start and Mode 3 SOS

Mode 2 uploads GPS positions and sample timestamps during the hike. The ground station checks the latest valid sample for a timeout and uses the uploaded position history when preparing a flight. Mode 3 stores the location and request information recorded when the user activates SOS, for operator verification and square-spiral route generation.

Mode 2 uses sampling time, not upload or receipt time. A stationary position with fresh samples does not trigger the current update-timeout rule. SOS uploads go to this system's Firebase database and ground station. The separate Call 999 Now button opens the phone dialer. The manuscript retains an earlier interface screenshot; the current English and Chinese SOS instructions explicitly describe ground-station operator verification.

Implementation: `M/src/services/eventBookingRecord.ts`. Booking interface: `M/src/pages/EventBookingPage.tsx`.

Reference: Ordnance Survey, Map Reading, "Timing", printed page 23. The complete source link is in `docs/Sources.txt`.

Mobile deployment configuration

The current mobile source supports Android only. For device operation, fill `M/src/services/db/firebaseConfig.ts` with the project's client configuration and use the same Realtime Database URL as the ground station. Configure the Android toolchain, native Firebase files and map credentials separately. From M, `npm start` starts Metro; `npm run android` invokes the Android build.

4 Ground-Station Verification and Dispatch

Common verification procedure

Overdue bookings, GPS update timeouts and new SOS requests enter the verification queue. The operator reads the user and emergency-contact details, contacts them and records the outcome. A confirmed safe status closes the alert. If safety cannot be confirmed, the operator contacts the available emergency contacts. After completing these records, the operator explicitly confirms a search before the platform creates a search event.

An SOS request no longer creates a dispatchable event directly. Repeated scans or confirmations do not create duplicates. A cancelled SOS cannot be confirmed through a stale alert. An unverified SOS event from the earlier implementation must complete contact verification before route preparation, provided it has not already entered execution.

Select an event, review its route and dispatch

After the operator selects an event, the ground station reads the associated user records and generates a route. Mode 1 uses the planned route; Mode 2 uses GPS history ordered by sample time; Mode 3 generates a square spiral around the SOS location. The operator reviews the route and settings before dispatch. Changes to the relevant records or operation state require the mission to be prepared again.

All three modes have the same initial event priority and the same waiting-time rule. SOS mode alone does not assign the highest priority. The operator selects and dispatches the mission.

Runtime configuration

Setting	Purpose
<code>GS_FIREBASE_DATABASE_URL</code>	Database URL shared with the mobile application

Setting	Purpose
GS_FIREBASE_CREDENTIALS	Local service-account file path
GS_T_LOCATION_UPDATE_SECONDS	Positive Mode 2 update timeout, in seconds
GS_T_WAIT_SECONDS	Positive queue waiting-time threshold, in seconds
GS_EVENT_TIMEZONE	Time zone for legacy bookings; template: Asia/Hong_Kong
GS_STATE_DIR	Local directory for dispatch journal files
Communication endpoint	Shared service address and port; see G/config/runtime.env.example

Use G/config/runtime.env.example as the template. Supply and export the actual values, then run python main.py from G. Both threshold fields are blank in the template.

Copy the template to G/config/runtime.env and fill it locally. The application reads exported environment variables. From the repository root in PowerShell:

```
Get-Content code\ground_station\config\runtime.env | ForEach-Object {
    if ($ -match '^s*([^\s]+)=(.*)$') {
        [Environment]::SetEnvironmentVariable($matches[1], $matches[2].Trim(), 'Process')
    }
}
Set-Location code\ground_station
python main.py
```

Both the ground station and UAV must reach the configured broker over the deployment network. Use the same database URL in the mobile and ground-station configuration.

Verification logic: G/src/rescue_event_manager.py. Persistence and mission preparation: G/src/rescue_repository.py, G/src/active_event_queue.py and G/src/ground_station.py.

5 UAV Software and External Flight Stack

Mission reception and execution status

The ground station and UAV exchange missions and status over Wi-Fi. A mission contains event and execution identifiers, a waypoint sequence, flight altitude, hover duration and return settings. The bridge validates the mission, passes it to the flight program and reports waypoint arrivals and mission stages to the ground station.

The phone-record receiver saves uploads received through its HTTP interface. After a matching mission reports completion of all waypoints and requests landing, the receiver forwards stored records to the ground station. The ground station saves each record and returns an acknowledgement matching its identifier and content hash. The UAV then marks that transfer as complete. The recording program saves downward-facing video on the onboard computer.

Location within U	Purpose
catkin_ws/src/rescue_bridge/src/	Mission communication and execution programs
catkin_ws/src/rescue_bridge/src/mission_commander.cpp	Waypoint and return-stage execution

Location within U	Purpose
drone_system/receiver/ phone_sos_receiver.py	Phone-record reception, storage and forwarding
drone_system/launcher/ cam_recorder.py	Onboard video recording
drone_system/launcher/ drone_console.py	Onboard program status and settings interface

Pinned external dependency

The flight stack uses Fast-Drone-250 from the original Drone repository at the commit below. The external dependency retains its own licenses and documentation.

b3ac1591b63d15270c9856e7b664da0cf9d3ea48

Download and placement commands are in Flight environment setup. Place the Fast-Drone-250 workspace, rescue_bridge catkin workspace and drone_system directory in the configured onboard workspace. Set directory paths, camera indices, serial ports and sensor calibration for that installation.

The ROS environment uses Noetic and catkin. Build the external flight workspace before the rescue_bridge workspace. The default flight altitude in rescue_bridge.launch is the original project setting of 5 m. The 80 m above-ground setting in Section 6 belongs to the separate MATLAB simulation.

Phone-receiver settings are in U/drone_system/config/rescue_receiver.env.example. Set UAV_RESCUE_TOKEN when exposing the interface on the local network. Keep database credentials, runtime tokens and actual user records in their respective deployment environments.

6 Three-Mode MATLAB Terrain Simulation

The simulation uses terrain near Pak Tam Chung, Sai Kung. The three modes share a launch location and a target in the hills, using a planned route, pre-dispatch GPS history and a square spiral around the target, respectively. Each mission includes takeoff, cruise, target-area activity, return and landing.

Setting	Value
Random seed	20260926
Launch location	22.402° N, 114.322° E
Target location	22.40782761° N, 114.35687745° E
Target ground elevation	Approximately 177.44 m
Cruise altitude above ground	80 m, following the terrain
Speed limits	Horizontal: 15 m/s; vertical: 3 m/s
Waypoint hover duration	5 s
Mode 3 spiral	Initial side length: 30 m; increase by 30 m every two segments; stop at the first endpoint at least 200 m from the centre

Run commands

Set the MATLAB current folder to simulation and select the required entry point:

```
run_all('paper')
run_all('simulate', fullfile(pwd, '..', 'local-results', 'simulation-01'))
run_all('video', fullfile(pwd, '..', 'local-results', 'video-01'))
```

paper redraws the figure from saved trajectories into a new directory under simulation/regenerated. simulate recomputes all modes into the selected output directory. video renders the saved trajectories into its selected directory and requires ffmpeg. Each output directory must be new; simulation/output remains the saved input and result archive. The renderers use Event Booking, Quick Start and SOS.

Mode	Input waypoints	Target arrival / s	Landing complete / s
Mode 1	3	360.66	715.65
Mode 2	15	409.08	764.07
Mode 3	19	344.99	1008.53

Elevation comes from the Mapzen / Tilezen Skadi N22E114.hgt tile, with a 3601×3601 grid at 1 arc-second spacing and EGM96 elevations. Bilinear interpolation gives a 25 m plotting grid without increasing source resolution. Routes and GPS history are synthetic inputs generated with the fixed seed. Times start at simulated takeoff. Parameters, inputs and executed trajectories are in simulation/output/scenario_and_settings.json, the per-mode waypoint CSV files and the execution_trace.csv files.

Mode 2 timestamps use 1 m/s to construct the synthetic walking history; this is separate from the mobile booking estimate of 4 km/h. All 15 samples exist before dispatch. Mode 3 visits the centre plus 18 spiral endpoints, completes the entire route and then returns; reaching the target does not terminate the search early.

7 System Videos and Outdoor Materials

Demo videos

System component	Video
SOS search pattern and end-to-end pipeline	DEMO — SOS Search Pattern End to End Pipeline
Onboard console and failsafe interface	DEMO — Drone Console On Board Failsafe GUI
Real-world test footage	DEMO — Real-World Test Footage

See System videos and outdoor materials for the video and material index.

Outdoor evidence

File in docs/assets/outdoor/	Contents
outdoor_flight.mp4	Original complete outdoor-test video
ground_station.mp4	Original ground-station operation video
airframe.jpg	UAV hardware photograph

File in docs/assets/outdoor/	Contents
launch_area.png	Original frame showing the launch area
search_area.png	Original frame showing the search area

The video files and images show the UAV, ground-station operation, launch site and search area. Numerical simulation trajectories and timestamps are stored in simulation/output.

References

Ordnance Survey's Map Reading guide provides the walking-time reference. Mapzen / Tilezen Skadi provides the elevation tile. The three original FYP GitHub repositories identify the upstream projects. Complete links are in docs/Sources.txt.

8 Implementation Index

M, G and U use the component abbreviations in Section 1. The following paths identify the source files implementing each system function.

User records and event creation

Event Booking uses M/src/services/eventBookingRecord.ts and M/src/pages/EventBookingPage.tsx for planned routes and trip-time estimation. G/src/rescue_event_manager.py detects overdue records and manages the contact procedure.

Quick Start uses M/src/pages/AndroidQuickStartPage.tsx and M/src/services/persistentTracking.ts. TrackingService, TrackingStore and TrackingCore in M/android/app/src/main/java/com/fypproject/tracking/ collect and persist samples. G/src/quick_start_freshness.py uses the latest valid sample time. Set GS_T_LOCATION_UPDATE_SECONDS for the deployment.

SOS upload and dialing are implemented in M/src/pages/SosPage.tsx. G/src/rescue_event_manager.py enforces contact verification and explicit search confirmation.

Route preparation and dispatch

G/src/ground_station.py and G/src/active_event_queue.py implement event selection, route preparation, review and confirmed dispatch. G/src/sos_pattern.py generates the square spiral. GS_T_WAIT_SECONDS controls the common waiting-time rule. G/src/dispatch_journal.py preserves mission transfer state across retries and restarts.

Mission execution and phone records

U/catkin_ws/src/rescue_bridge/src/ contains the mission communication program and mission_commander.cpp, which advances navigation goals. A ROS build requires the environment in docs/Flight_Environment_Setup.txt.

U/drone_system/receiver/phone_sos_receiver.py and G/src/rescue_record_protocol.py receive, persist, return and acknowledge phone records.

Simulation

simulation/run_three_mode_simulation.m constructs the fixed-seed scenario and full trajectories. simulation/output/ holds inputs, events, execution traces, mission_summary.csv and three_mode_study.mat. simulation/render_paper_terrain.m renders the manuscript's terrain figure from saved data.